

Liftoff X — RGS & Seamless Wallet Architecture

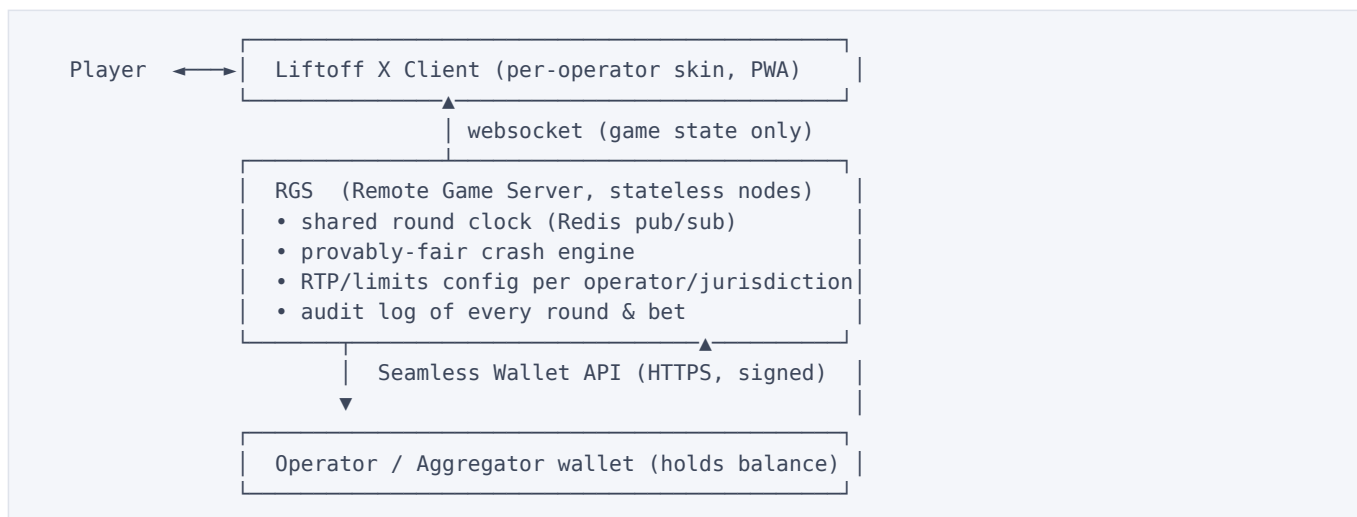
How we turn the MVP into an operator-integratable product. This is the tech direction to lock before building further, so the work counts toward certification and aggregator listing rather than being thrown away.

1. Where the MVP is today

Layer	MVP (today)	File
Game server	Socket.io, authoritative shared round clock, provably-fair crash	`server/game-server.mjs`
Wallet	In-process demo wallet (guest JSON / Supabase)	`server/store.mjs`
Fairness	HMAC-SHA256, client-verifiable	`verify-fairness.mjs`
Client	Next.js + canvas engine, same-origin	`app/`

The **game-logic core is already the right shape**: server-authoritative rounds, a clean round lifecycle (**betting** → **start** → **crash**), provably-fair crash points, and per-player bet/cashout settlement. The piece that must change for B2B is the **wallet**: in B2B the operator owns the player's money, not us.

2. Target architecture



Key change: **the RGS never holds player funds**. For every bet it debits the operator wallet; for every cashout it credits it. The operator is the source of truth for balance. This is what every aggregator integration requires.

3. Seamless ("transactional") wallet API

The RGS calls **the operator's** endpoints. All calls: HTTPS, **idempotent**, signed (HMAC over body + timestamp), with a unique **transactionId** and a **roundId/betId** for reconciliation.

Call	When	Idempotency key
`authenticate`	session start (token → playerId, currency, balance)	session token
`balance`	on demand / reconnect	—
`bet` (debit)	player places a bet	`betId`
`win` (credit)	cashout settles	`betId` (+ `txId`)
`rollback`	failed/duplicate bet, round void	original `txId`

Example — `bet` (debit)

```
POST {operator.walletUrl}/bet
{
  "operatorId": "acme",
  "playerId": "p_8842",
  "currency": "EUR",
  "amount": 1.00,
  "gameId": "liftoffx",
  "roundId": "rnd_000123",
  "betId": "bet_000123_p8842_s0", // slot 0 of the dual-bet
  "transactionId": "tx_9f3a...", // unique, idempotent
  "timestamp": 1730000000,
  "signature": "hmac-sha256(...)"
}
// → 200 { "balance": 124.00, "currency": "EUR", "txId": "op_tx_55" }
// → 402 { "code": "INSUFFICIENT_FUNDS" } // reject the bet
```

Example — `win` (credit on cashout)

```
POST {operator.walletUrl}/win
{
  "betId": "bet_000123_p8842_s0",
  "roundId": "rnd_000123",
  "amount": 2.81, // gross payout (stake × multiplier)
  "multiplier": 2.81,
  "transactionId": "tx_a17c...",
  "signature": "..."
}
// → 200 { "balance": 126.81 }
```

Rule: a bet that was debited but never settled (crash = loss) needs no credit — the debit *is* the loss. Only cashouts credit. A bet that fails after debit must `rollback`. Every `bet`/`win` carries the `betId` so the operator can reconcile 1:1.

Mapping to the existing round lifecycle

`server/game-server.mjs` already has the hooks — they just change target:

MVP today	B2B RGS
`bet:place` → `p.rec.balance -= amount`	→ `bet` (debit) operator wallet; on `402` emit `bet:rejected`
`cashout` → `settleCashout` credits local balance	→ `win` (credit) operator wallet
crash busts un-cashed bets	→ no wallet call (debit already taken)
`pushProfile` balance	→ balance comes from wallet responses

4. Config service (per operator / jurisdiction)

Each operator integration carries config the RGS enforces:

- **RTP version** (97/96/95) — selects the house-edge slice in the crash engine.

- **Min/max bet, max win cap, currency** rounding.
- Enabled features (chat, social feed) — some jurisdictions disable chat.
- Responsible-gaming limits passed through from the operator.

(Today the house edge is a constant in `verify-fairness.mjs` — make it a per-round parameter sourced from operator config, and keep the provably-fair proof intact.)

5. Fairness & certification

- Keep **provably fair** (player verification) as a product feature.
- For regulated markets add **certified RNG (GLI-19/GLI-11)**: documented seeding, server-seed rotation, and a tamper-evident **audit log** of every round (serverSeed, hash, nonce, crash point, all bets/settlements).
- Reconciliation report per operator per day (turnover, GGR, our share).

6. Scale & availability

- Make RGS nodes **stateless**; hold the shared round clock + live state in **Redis** (pub/sub for round events, so any node can serve any player).
- Sticky sessions **not** required if all round state is in Redis.
- Targets: low round-tick latency, horizontal scale per region, graceful reconnect (snapshot the current round on connect — already done in the MVP via `snapshotFor`).

7. Migration path (MVP → RGS) — concrete steps

- **Extract** the game core (`attachGame`) from the Next single-origin server into a standalone **RGS service**; client connects to it per operator.
- **Abstract the wallet** behind an interface (it already is, via `makeStores`): add a `WalletStore` implementation that calls the **seamless wallet API** instead of mutating a local balance.
- **Parameterise RTP/limits** from per-operator config (replace the constant edge).
- **Move shared state to Redis** (round clock, players, activity) for multi-node.
- **Add the audit log + daily reconciliation** export.
- **Harden**: request signing, idempotency, rate limits, anti-collusion checks.
- **Certification dry-run** with a test lab's checklist (GLI/iTech) before listing.
- **Aggregator adapter**: implement one aggregator's wallet/GAP spec first (e.g. Hub88/SoftSwiss) — that single integration unlocks many operators.

Order of value: steps 1-3 make it **integratable**; 4-6 make it **production-grade**; 7-8 make it **sellable* in regulated markets*.